

OpenShift GitOps in pratica

Capitolo 1

Perche Docker non basta piu

Laboratorio 100% localhost-first

Nessun cloud obbligatorio. Gli esercizi si eseguono sul PC del lettore con Docker/Podman e, nei capitoli successivi, OpenShift Local/OKD.

Stack del libro: React, Spring Boot, PostgreSQL, Docker/Podman, Kubernetes, OpenShift, Tekton, Argo CD, GitHub webhook.

Versione formattata - Capitolo 1

Indice del capitolo

- 1.1 Il progetto BookStore
- 1.2 Perché non partiamo subito da OpenShift?
- 1.3 Container: cosa ci serve sapere davvero
- 1.4 Primo ambiente locale con Docker Compose
- 1.5 File docker-compose.yml
- 1.6 Cosa significa questo file
- 1.7 Dockerfile del backend Spring Boot
- 1.8 Dockerfile del frontend React
- 1.9 Avvio dell'ambiente
- 1.10 Prima verifica
- 1.11 Networking: perché il backend usa postgres e non localhost
- 1.12 Persistenza: perché serve un volume
- 1.13 Esercitazione 1 - avviare e fermare tutto
- 1.14 Simuliamo un crash
- 1.15 Simuliamo lo scaling
- 1.16 Simuliamo un aggiornamento
- 1.17 Dove Docker Compose è perfetto
- 1.18 Dove Docker Compose non basta
- 1.19 Perché arriva Kubernetes
- 1.20 Cosa hai imparato
- 1.21 Errori comuni
- 1.22 Esercizio finale svolto su localhost
- 1.23 Collegamento al capitolo 2

Capitolo 1 - Perché Docker non basta più

In questo capitolo partiamo dal punto più familiare per uno sviluppatore: far funzionare una piccola applicazione composta da frontend, backend e database sul proprio PC. Non useremo ancora Kubernetes, OpenShift, Tekton o Argo CD. Li introdurremo solo quando il problema li renderà necessari.

Obiettivo pratico - alla fine del capitolo avrai una struttura di progetto BookStore eseguibile su localhost con Docker Compose, più una lista chiara dei limiti che ci porteranno a Kubernetes e OpenShift.

1.1 Il progetto BookStore

Useremo sempre la stessa applicazione in tutto il libro. Questo evita esempi scollegati e permette di vedere l'evoluzione reale di un progetto da semplice applicazione locale a pipeline GitOps completa.

```
bookstore/  
├── frontend/      # applicazione React  
├── backend/       # API Spring Boot  
└── database/      # script SQL iniziali
```

```

└─ infrastructure/
   └─ docker/      # Docker Compose e file locali
   └─ kubernetes/  # manifest Kubernetes
   └─ openshift/   # Route, Project, SCC se necessarie
   └─ tekton/      # Pipeline, Task, Trigger
   └─ argocd/      # Application e progetti GitOps
  └─ README.md

```

Nel primo capitolo useremo solo Docker Compose. Il frontend sarà raggiungibile da browser su localhost, il backend esporrà API HTTP sempre su localhost e PostgreSQL rimarrà confinato alla rete locale Docker.

1.2 Perché non partiamo subito da OpenShift?

Perché OpenShift risolve problemi che diventano evidenti solo dopo aver provato a gestire container, configurazioni, reti, crash, aggiornamenti e immagini. Se partissimo subito da OpenShift, sembrerebbe una piattaforma enorme piena di oggetti YAML. Se invece partiamo da Docker, ogni oggetto successivo avrà un motivo.

1.3 Container: cosa ci serve sapere davvero

Un container è una istanza in esecuzione di una immagine. Una immagine è un pacchetto immutabile che contiene applicazione, runtime e dipendenze. Questo ci permette di evitare il classico problema: "sul mio PC funziona, sul tuo no".

La cosa importante per uno sviluppatore è questa: il container non è una piccola macchina virtuale. Condivide il kernel dell'host e isola processi, filesystem, rete e risorse. Questo lo rende più leggero di una VM, ma anche meno adatto a simulare un intero datacenter da solo.

1.4 Primo ambiente locale con Docker Compose

Docker Compose permette di descrivere più servizi in un unico file YAML: servizi, reti, volumi, variabili ambiente e porte. La documentazione Docker lo presenta proprio come lo strumento per definire e avviare applicazioni multi-container con un singolo comando.

Nel nostro laboratorio useremo Compose per eseguire frontend, backend e database.

1.5 File docker-compose.yml

Crea il file seguente in `infrastructure/docker/docker-compose.yml`.

```

services:
  postgres:
    image: postgres:17
    container_name: bookstore-postgres
    environment:
      POSTGRES_DB: bookstore
      POSTGRES_USER: bookstore
      POSTGRES_PASSWORD: bookstore
    ports:
      - "5432:5432"
    volumes:
      - bookstore_pgdata:/var/lib/postgresql/data
    networks:
      - bookstore-net

  backend:
    build:
      context: ../../backend
      dockerfile: Dockerfile
    container_name: bookstore-backend
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/bookstore

```

```

    SPRING_DATASOURCE_USERNAME: bookstore
    SPRING_DATASOURCE_PASSWORD: bookstore
  ports:
    - "8080:8080"
  depends_on:
    - postgres
  networks:
    - bookstore-net

frontend:
  build:
    context: ../../frontend
    dockerfile: Dockerfile
  container_name: bookstore-frontend
  environment:
    VITE_API_BASE_URL: http://localhost:8080
  ports:
    - "3000:80"
  depends_on:
    - backend
  networks:
    - bookstore-net

volumes:
  bookstore_pgdata:

networks:
  bookstore-net:
    driver: bridge

```

1.6 Cosa significa questo file

- postgres usa una immagine ufficiale e conserva i dati in un volume locale.
- backend viene buildato dalla cartella backend e raggiunge PostgreSQL usando il nome del servizio postgres.
- frontend viene buildato dalla cartella frontend e viene esposto su localhost:3000.
- bookstore-net è una rete Docker bridge: i container possono parlarsi usando i nomi dei servizi.
- Le porte pubblicate servono solo per accedere dall'host: browser, curl, strumenti SQL.

Nota localhost - il database è pubblicato su localhost:5432 solo per comodità didattica. In un ambiente reale potresti evitare di pubblicarlo fuori dalla rete interna dei container.

1.7 Dockerfile del backend Spring Boot

Nel backend useremo una build semplificata. In un progetto reale potresti usare Maven Wrapper e una build multi-stage. Qui preferiamo un Dockerfile leggibile.

```

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/bookstore-backend.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

Questo Dockerfile assume che il JAR sia già stato costruito localmente. Nei capitoli su Tekton sposteremo la build dentro la pipeline.

1.8 Dockerfile del frontend React

```

FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html

```

Il frontend viene compilato con Node e poi servito da Nginx. Questo schema è molto comune: il container finale è leggero e contiene solo file statici.

1.9 Avvio dell'ambiente

```
cd bookstore/infrastructure/docker
docker compose up -d --build
```

Dopo qualche secondo dovresti poter aprire:

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:8080>
- PostgreSQL: <localhost:5432>

1.10 Prima verifica

```
docker compose ps
curl http://localhost:8080/actuator/health
```

Se il backend espone Spring Boot Actuator, la risposta attesa sarà simile a:

```
{"status": "UP"}
```

1.11 Networking: perché il backend usa postgres e non localhost

Dentro un container, localhost indica il container stesso. Se il backend prova a collegarsi a localhost:5432, cerca PostgreSQL dentro il container backend, non dentro il container postgres. Per questo usiamo jdbc:postgresql://postgres:5432/bookstore:postgres e il nome DNS interno del servizio Compose.

```
backend container
localhost:8080 -> backend stesso
postgres:5432 -> container PostgreSQL sulla rete bookstore-net
```

1.12 Persistenza: perché serve un volume

Senza volume, i dati del database sparirebbero quando il container viene eliminato. Il volume bookstore_pgdata conserva i file di PostgreSQL sul disco dell'host, ma li monta dentro il container nel percorso atteso da PostgreSQL.

```
volumes:
  bookstore_pgdata:

postgres:
  volumes:
    - bookstore_pgdata:/var/lib/postgresql/data
```

1.13 Esercitazione 1 - avviare e fermare tutto

1. Avvia l'ambiente con `docker compose up -d --build`.
2. Controlla i container con `docker compose ps`.
3. Apri il frontend su <http://localhost:3000>.
4. Ferma tutto con `docker compose down`.
5. Riavvia con `docker compose up -d` e verifica che il database conservi i dati.

1.14 Simuliamo un crash

Ora rompiamo apposta il backend.

```
docker stop bookstore-backend  
docker compose ps
```

Con Compose il container risulta fermo. Puoi aggiungere `restart: unless-stopped`, ma stai ancora dicendo a Docker cosa fare su una singola macchina. Non stai descrivendo un cluster, non stai distribuendo il carico, non stai facendo rollout controllati tra nodi diversi.

Problema - Docker può riavviare un container sullo stesso host. Kubernetes invece ragiona in termini di stato desiderato del cluster: voglio N repliche sane, indipendentemente dal nodo su cui girano.

1.15 Simuliamo lo scaling

Proviamo a scalare il backend.

```
docker compose up -d --scale backend=3
```

Qui incontrerai subito un problema se il servizio backend pubblica la porta 8080: tre container non possono usare tutti la stessa porta host 8080. Dovresti rimuovere il mapping fisso, mettere un reverse proxy o cambiare architettura.

Questo è uno dei momenti in cui Docker Compose mostra il suo limite didattico: va benissimo per sviluppo locale, ma non è un orchestratore pensato per gestire automaticamente repliche, bilanciamento e rollout in un cluster.

1.16 Simuliamo un aggiornamento

Immagina di avere backend versione 1.0 e di voler passare alla 1.1 senza downtime. Con Compose puoi fare `rebuild` e `restart`, ma il controllo è limitato. Kubernetes introduce Deployment e rollout proprio per gestire aggiornamenti graduali e rollback.

1.17 Dove Docker Compose è perfetto

- Sviluppo locale.
- Applicazioni piccole.
- Test rapidi di frontend, backend e database.
- Demo e ambienti temporanei.
- Onboarding di un progetto su una singola macchina.

1.18 Dove Docker Compose non basta

- Alta disponibilità tra più nodi.
- Scheduling automatico dei container.
- Service discovery stabile in un cluster.
- Rollout e rollback controllati.
- Policy di sicurezza, RBAC e gestione multi-team.
- Separazione reale di ambienti dev, staging e prod.
- GitOps e riconciliazione continua dello stato desiderato.

1.19 Perché arriva Kubernetes

Kubernetes entra in scena quando non vuoi più dire: "avvia questo container qui". Vuoi dire: "questo è lo stato desiderato: tre repliche del backend, una porta stabile per raggiungerle,

configurazione separata dal codice, rollback possibile". Kubernetes osserva lo stato reale e lo confronta con lo stato desiderato.

La documentazione Kubernetes descrive questa idea con i Deployment: dichiarare lo stato desiderato e il Deployment Controller cambia gradualmente lo stato reale per raggiungerlo. Questo è il concetto che useremo per arrivare naturalmente a OpenShift.

1.20 Cosa hai imparato

- Docker impacchetta ed esegue applicazioni in container.
- Docker Compose semplifica applicazioni multi-container su localhost.
- I container sulla stessa rete Compose si raggiungono usando il nome del servizio.
- I volumi servono per conservare dati oltre la vita del container.
- Crash, scaling, rollout e ambienti multipli sono il punto in cui serve un orchestratore.
- Kubernetes non sostituisce Docker: risolve un problema diverso.

1.21 Errori comuni

- Usare localhost dentro un container per raggiungere un altro container.
- Dimenticare i volumi e perdere i dati del database.
- Pubblicare su host porte che dovrebbero restare interne.
- Confondere Compose con una piattaforma di produzione.
- Pensare che restart: always equivalga ad alta disponibilità.

1.22 Esercizio finale svolto su localhost

Obiettivo: verificare concretamente i limiti di Compose.

1. Avvia BookStore con `docker compose up -d --build`.
2. Ferma il backend con `docker stop bookstore-backend`.
3. Verifica che il frontend non riesca più a chiamare le API.
4. Riavvia con `docker compose up -d backend`.
5. Prova a scalare il backend a 3 repliche.
6. Osserva il conflitto sulla porta 8080 oppure modifica il compose per non pubblicare la porta del backend.
7. Scrivi nel README quali problemi hai incontrato e perché Kubernetes li risolve meglio.

1.23 Collegamento al capitolo 2

Nel prossimo capitolo prenderemo la stessa applicazione e la porteremo su Kubernetes locale. Non useremo ancora OpenShift. Prima vogliamo vedere i concetti nativi: Pod, Deployment, Service e Namespace. Poi, solo dopo, capiremo perché OpenShift rende tutto più adatto a un team di sviluppo reale.

Fonti essenziali del capitolo

Riferimenti primari usati per mantenere il capitolo allineato alla documentazione ufficiale. I link sono cliccabili nel DOCX e nel PDF esportato.

Fonte	Per cosa viene usata	Link
Docker Docs	Compose: Concetti e uso di Docker Compose per applicazioni multi-container.	https://docs.docker.com/compose/
Docker Docs	Compose file reference: Formato del file Compose: servizi, reti, volumi e configurazione.	https://docs.docker.com/reference/compose-file/
Docker Docs	Dockerfile reference: Istruzioni Dockerfile come FROM, COPY, RUN, EXPOSE ed ENTRYPOINT.	https://docs.docker.com/reference/dockerfile/
Docker Docs	Spring Boot samples: Esempi Docker con backend Spring Boot, frontend React e database.	https://docs.docker.com/reference/samples/spring/
Spring Boot Docs	Container Images: Opzioni ufficiali per containerizzare applicazioni Spring Boot.	https://docs.spring.io/spring-boot/reference/packaging/container-images/index.html
Spring Guide	Spring Boot with Docker: Guida introduttiva alla creazione di immagini Docker per applicazioni Spring Boot.	https://spring.io/guides/gs/spring-boot-docker
React Docs	Build a React app from Scratch: Indicazioni ufficiali React sull'uso di build tool come Vite.	https://react.dev/learn/build-a-react-app-from-scratch
Vite Docs	Getting Started: Guida ufficiale Vite per creare e avviare applicazioni frontend moderne.	https://vite.dev/guide/
Kubernetes Docs	Overview: Kubernetes come piattaforma open source per workload containerizzati, configurazione dichiarativa e automazione.	https://kubernetes.io/docs/concepts/overview/
Kubernetes Docs	Deployments: Deployment, ReplicaSet, stato desiderato e aggiornamenti dichiarativi.	https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
Kubernetes Docs	Services: Service come endpoint stabile davanti a uno o più Pod.	https://kubernetes.io/docs/concepts/services-networking/service/

Nota: nei capitoli successivi verranno aggiunte fonti specifiche per Kubernetes, OpenShift, Tekton, Argo CD e GitOps.